

CS6310 – Software Architecture & Design

Assignment #1 (100 points): Pokémon Tournament Project – Phase 1 (v0)

"Welcome to the Poke-Dome!"

Spring 2025 Semester – created by the CS6310 Instructional Team

Submission Deliverables

- This assignment must be completed as part of an approved group.
- You must submit the following items in Canvas:
 - (1) A PDF file named **class_diagram.pdf** that contains your UML Class (Model) Diagram representing the static structure of the classes, attributes, methods, and the relationships between the (class-generated) objects.
 - (2) A PDF file named **sequence_diagram.pdf** that contains your UML Sequence Diagram representing the dynamic sequence of actions for the object interactions specified below.
- You may divide your class model diagram or sequence diagrams into multiple files if needed. For example, if the initial diagram is so large that dividing it into multiples makes it significantly easier to read, then you may submit multiple files with reasonable name extensions such as **class_diagram1.pdf**, **class_diagram2.pdf**, etc. You must provide enough context for the multiple diagrams to allow us to connect them and understand the singular/completed diagram.
- You must notify us via a private post on Canvas and/or Ed Discussion BEFORE the Due Date if you are encountering difficulty submitting your project. You will not be penalized for situations where Canvas is encountering significant technical problems. However, you must alert us before the due date – not well after the fact. You are responsible for submitting your answers on time in all other cases.
- Uploading files to Canvas occasionally takes a significantly long time, especially based on the relative size of the files being submitted. Please plan accordingly, as submissions outside of the Canvas Availability Date will likely not be accepted. You are permitted – and highly recommended – to take advantage of the opportunity to submit your files an unlimited number of times. Use the same file naming standards for your (optional) "interim submissions" that you do for the final submission.

Problem Scenario

You are being asked to design and develop a system to manage battles between Pokémon. For those unfamiliar with this world, Pokémon represent "pocket monsters" – fictitious creatures that have special powers and co-exist with humans. Humans collect and care for these creatures, and – on occasion – pit these creatures against each other in "battles." Your system must support the capability to allow Pokémon to battle each other, and to keep track of the status of the creatures to include (but not limited to) their capability levels, health, and battles conducted (i.e., wins/losses). And your system must allow new and arbitrary Pokémon to be defined dynamically.

There are various websites that you can use to learn more about the Pokémon universe, including:

- <https://www.pokemon.com/us>
- <https://en.wikipedia.org/wiki/Pokemon> [alt: <https://en.wikipedia.org/wiki/Pok%C3%A9mon>]
- <https://pokemongolive.com/?hl=en>

You do not have to be a “Pokémon master” to complete this project successfully, but understanding something about the series will be helpful. The assignment instructions will specify the requirements for your system.

Evaluation/Grading Your Submissions

Your submission will be evaluated based on these key factors:

[1] UML Class (Model) Diagram [~50% of the assignment]

You must provide a Class Diagram as a file named **class_diagram.pdf** that includes:

- classes (a minimum of 3 classes, a maximum of 10 classes)
- attributes with basic types
- operations (names only – methods, as the implementations of the operations, will be required later during the development process)
- relationships with proper cardinalities (e.g., [0..*]) that accurately reflect the problem space
- visibility notation (e.g., public, private, etc.)

To make your class diagram more readable, you may omit very basic “setters and getters” [e.g., set_() and get_() based operations and methods], especially those of the one-line variety that only provide simple and direct read or write access for an object attribute. Further details about the requirements for your Class Diagram can be found in the UML Guidelines on Canvas.

[2] UML Sequence Diagrams [Two sequence diagrams, each ~25% of the assignment]

You must provide distinct Sequence Diagrams as files named **sequence_diagram#.pdf** that include:

- lifeline boxes representing participants: somewhat like instantiated objects of a certain class, but can also represent external actors in some instances
- vertical lifeline for each participant, usually indicating the actions of that participant over time
- messages between objects with names and parameters, and (explicit) replies/return messages
- instances of object/participant creation
- design frames representing selection (**alt**), iteration (**loop**), and more complex control flow structures if needed (e.g., **par** for parallel execution, or **region** for critical areas of the system normally protected with mutexes, semaphores, etc. to restrict execution to a single thread)
- (optional) “focus of control” and “execution specification” bars (that demonstrate flow of control issues – for example, whether an object will wait synchronously for a response, or will continue processing asynchronously after sending a message)

For your Sequence Diagrams, you only need to show the interactions required to successfully complete the following two types of event sequences per your design:

- **One complete turn of a battle where Pokémon X attacks Pokémon Y and Pokémon Y does NOT defend itself**
- **One complete turn of a battle where Pokémon Y attacks Pokémon X and Pokémon X does defend itself**

You must create enough notional objects to demonstrate the messages and related data being shared to complete each of these sequences of events successfully. These actions might occur as one continuous flow during execution, but we require two (2) distinct diagrams for clarity.

One of the significant issues that will cause you to lose points with these diagrams is submitting Class Model & Sequence Diagrams that are inconsistent, and/or that do not reflect the latest changes to the problem requirements. For example, messages sent between objects in your sequence diagram must correspond to the operations and methods as shown in the class diagram.

Overall Project Timeline

The course project will be divided into two phases, and you'll work together with your peers in groups of three, four or five designers. For the first phase of the project, you will be focusing on items [1] & [2], for the second phase of the project you will be focusing on all 3 parts.

[1] Analyze the most up-to-date problem description and requirements

Each phase requires you to review and analyze the provided requirements and specifications to identify the entities, attributes, operations, methods, and relationships that are key to the successful design and development of the system. You will not receive all the project details during the first phase of the project. You will receive instructions during the later phases that will supplement and/or override the earlier information. You must design your system in a way that will allow it to evolve as gracefully as possible.

Sometimes, the new information might seem unclear, ambiguous, or even contradictory to the existing facts. And you might not realize these conflicts immediately upon reading them, but only after you've attempted to implement some of these changes in your system. In these cases, you may seek clarification from the instructional team. After receiving further guidance, you should make any other reasonable assumptions (e.g., that don't conflict with established requirements) as needed to complete the assignment.

[2] Create and/or update the required design artifacts

Each phase requires you to provide one or more original and/or updated design artifacts. All design artifacts will be selected from the various Unified Modeling Language (UML) diagrams, like the Object-Oriented Analysis (OOA) discussed in videos P2L1 – P2L3 and demonstrated in P2L5.

[3] Implement and test source code that supports the required functional capabilities

The latter phases require you to create source code that implements the required capabilities. The earlier phases will focus on the design requirements, while the later phases will integrate functional and non-functional requirements as well. We will assess the correctness of your system from three main perspectives:

- Does your system display solid (object-oriented) design principles and quality?
- Is your source code organization consistent with the design artifacts that you provided?
- Which of the complete set of test cases does your system pass successfully?

We will often provide you with an initial and limited set of test cases, but we often reserve some test cases for the official testing of each phase. And there might also be a few additional artifacts that are required in the later phases: for example, a video demonstration of the final version of your system.

You are allowed – and strongly encouraged – to take an iterative approach: don't feel that you must generate a single "perfect" set of designs on your first attempt. Instead, generate an initial (or updated) set of designs, then implement source code based on those designs, and then review your results to determine if you're on the right track. Continue to modify your designs and/or implementation details until you've successfully met all the requirements; or, as many requirements as possible before the submission deadline.

Don't lose sight of the goal for the course: there are many more potential improvements that you could make to this system, but you really need to think about how you will separate and distribute the complexity of the overall problem across separate classes; and, how the objects instantiated from those classes then communicate and collaborate to solve the problem. Helping you develop a solid structure for your design is the most important aspect of working on this assignment.

Next: a More Detailed Description of Pokémon...

The Pokémon for our system must be implemented in a way that allows them to reference and interact with each other. They have certain characteristics, some of which are permanent, and others which change over time. We are going to use a specific subset of these characteristics for our system, so – to all the Pokémon enthusiasts – please be patient if we don't incorporate all the details of the Pokémon universe.

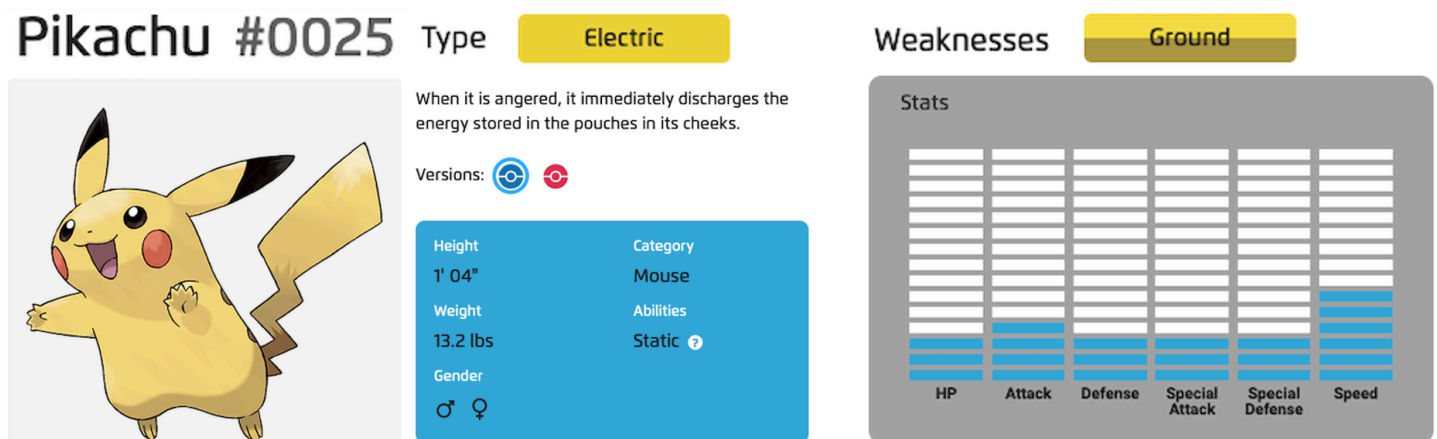


Figure 1: Pikachu from the Pokémon Pokédex (i.e. catalog), <https://www.pokemon.com/us/pokedex/pikachu>, 15 January 2024

First, Pokémon have permanent names (i.e., as a string) like Pikachu (shown above), Bulbasaur, or Squirtle. They also have an attribute called hit points (i.e., as an integer) that measures the health of the creature. The Pokémon begin the battles at full strength (i.e., as an integer). The Pokémon also have varying combinations of offensive and defensive skills, and the Pokémon must use these skills while battling each other. During the battles, a Pokémon's hit points are reduced when it has been attacked successfully. The battle continues until one of the Pokémon reaches zero hit points. Your system must coordinate the interactions between the two battling Pokémon much like a sports referee or game arbiter.

A key aspect of your system is that new Pokémon will be introduced to your system as objects for testing purposes after submission (Think of it like adding downloadable content to your pre-existing game). Your system will need to interact with the new Pokémon objects dynamically. Each Pokémon object must manage certain aspects of its own state and behavior – for example, each Pokémon must:

- Be able to initialize itself as needed and prepare itself for battle
- Manage its own health/hit points
- Be aware of its attack skills and its defensive skills
- Identify which attack and/or defense it will use for a given round while battling another Pokémon
- Determine how the damage levied by an attack against it will affect its health – for example, will a prior defensive action minimize the damage

You should not “hardcode” these effects into your system – rather, your system must let the Pokémon manage those aspects themselves. Also, even though we’ve shown you examples of “classic” Pokémon that have already been created, be aware that we will introduce new Pokémon when testing your system that will have different health levels and types of attacking and defensive skills.

Given the need to handle dynamic, run-time classes and objects, we strongly recommend that you investigate Java Reflection, and consider it as an approach (or part of your approach) to developing your system. Here are some sites that might be helpful:

<https://www.oracle.com/technical-resources/articles/java/javareflection.html>

<https://docs.oracle.com/javase/tutorial/reflect/>

https://download.java.net/java/early_access/panama/docs/api/java.base/java/lang/reflect/package-summary.html

...Followed by a More Detailed Description of the Battle Process

Pokémon battles are conducted with a specific process. Currently, the first Pokémon - **<Pokemon_1>** in the **battle,<Pokemon_1>,<Pokemon_2>** command - gets the opportunity to attack **<Pokemon_2>** first. Then, **<Pokemon_2>** is given an opportunity to attack **<Pokemon_1>**. These opportunities to attack each other continue to alternate between the two battling Pokémon until one of the Pokémon reaches zero hit points. That Pokémon passes out and loses the battle; then, both Pokémon are given a chance to rest up and return to their maximum hit point in preparation for the next battle.

A Pokémon determines what it will do – attack or defend – based on the ratio of its current hit points divided by the maximum number of hit points:

- If the ratio is greater than or equal to 0.7, then the Pokémon is still relatively strong, and will be very aggressive. In this case, there is an 80% likelihood that the Pokémon will decide to attack.
- If the ratio is less than 0.7, but still greater than or equal to 0.3, then the Pokémon is noticeably weaker than the start of the battle and will act in a more balanced manner. In this case, there is a 50% likelihood that the Pokémon will decide to attack.

- Finally, if the ratio is less than 0.3, then the Pokémon is significantly weaker and will act much more cautiously. In this case, there is only a 30% likelihood that the Pokémon will decide to attack.
- You can think of these thresholds like generating a random number, then doing a check on that number. If you are 80% likely to attack, then If there was 10 numbers that your random number generator could produce, and it generated anything greater than or equal to 2, you would attack, the remaining 0 & 1 values if selected would mean you defend.

In all these cases, if the Pokémon does not decide to attack, then it will defend itself. When the Pokémon attacks, then your system must select one of the Pokémon's attack skills randomly with an even likelihood for all the possible skills (i.e., a uniform probability distribution). Similarly, if the Pokémon decides to defend itself instead of attacking, then your system must select one of the Pokémon's defensive skills randomly with an even likelihood for all the possible skills.

Attacking skills and defensive skills are all identified by a name and a damage level. When a Pokémon launches an attack, your system must consider the level of damage that might be inflicted on the Pokémon being attacked. However, if the Pokémon being attacked selected a defensive skill on its prior action, then the full damage from the attacking skill might be limited – or even eliminated – by the defensive skill.

- If there was a defensive action on the prior turn, then the limited damage (i.e., full attack damage level – defensive damage level) is assessed.
 - If there was no defensive action on the prior turn, then the full attack damage level is assessed.
- After the attacking and defensive actions have been assessed, and the attacked Pokémon's hit points have been updated appropriately, your system must confirm that the Pokémon still has hit points. Finally, all fractional damage values must be rounded up to the next integer value (i.e., ceiling).

Randomness and Simulation Consistency

There are a number of moments during the **battle()** processing that require decisions to be made based on a random value. Your system must use Java's Random capabilities to manage and generate all random values that are needed. To support a pseudo-random but consistent sequence of values, the Random library allows you to use a seed value to initialize the value generation process. This supports the unpredictable behavior of Pokémon (to make things fun & interesting) while also allowing us to identify errors and other unexpected actions consistently.

Situations where Random Values must be Generated and Used in a Specific Manner:

- **Controlling the Pokémon's Actions:** Each Pokémon will make several decisions during a battle. Consequently, each Pokémon must maintain a special "seed" value (as an integer) that must be used – once provided by the user/test case – to initialize its random value sequences. And if the **setseed,<Value>** command is used, then **<Pokemon_1>** must use **<Value>** as its random seed while **<Pokemon_2>** must use **<Value + 1>** as its random seed. This ensures that the Pokémon do not completely "mirror" (i.e., duplicate) each other's actions, which makes the

battles much more interesting. This seed will remain until the program is stopped, or you remove it with the remove command.

- **Determining if a Pokémon will Attack or Defend:** A Pokémon will vary its likelihood to attack versus defending itself (e.g., “aggressiveness”) based on its current hit points. This likelihood (i.e., decision) to attack is then given as a probability, and your system must generate a random integer value between 0 to 9 (inclusively). It must attack if – and only if – the value is greater than or equal to the appropriate likelihood (i.e., “aggressiveness” percentage).

Suppose that the Pokémon has a 60% likelihood of attacking. If the random value generated is between 4 and 9 (inclusive), then the Pokémon must attack; otherwise, if the value is 3 or lower, then the Pokémon must defend itself instead.

- **Selecting a Pokémon’s Next Attack or Defense Skill:** When selecting the next offensive or defensive skill for a Pokémon, your system must generate an appropriately sized random integer value based on the number of possible options, and then select the skill that corresponds to that index/position in the list, and the list of skills must be sorted in ascending order based on the damage level for each skill.

Suppose that a Pokémon has four attacks: (Aeroblast, damage: 100), (Fury Attack, damage: 15), (Meteor Assault, damage: 150), and (Shadow Punch, damage: 60). If the random value selected is 2, then the attack that is selected must be Aeroblast.

And the **removeSeed()** command must nullify (i.e., remove) the existing seed values from the Pokémon. Your system must support a sequence of **setSeed()** → **removeSeed()** → **setSeed()** commands before the battle commences, and the latest seed value must be used correctly. Finally, the **stop** command should cause a graceful exit of the command input loop.

Walkthrough of an Example Test Case

This is an example of the test case results, with test case lines interspersed with the expected output.

Welcome to the thunder dome!

```
> // set the seed to 1
```

```
> setseed,1
```

The “**Welcome to the thunder dome!**” message must be displayed at the beginning of each simulation run. The lines that begin with the “>” character are either comments or commands from the test case. Comments are indicated with the “//” delimiter and included in the test case to add clarity. And the **setseed,1** command sets the random seed value to 1 initially.

```
> // remove the seed
```

```
> removeSeed
```

The **removeSeed** command removes the effect of setting the setting the seed to the value 1.

```
> // set the seed to 10
```

```
> setseed,10
```

This (second) **setseed,10** command sets the random seed value to 10.

```
> // battle between Bulbasaur & Pikachu
```

```
> battle,Bulbasaur,Pikachu
```

This command initiates a battle between Bulbasaur as <Pokemon_1> and Pikachu as <Pokemon_2>. From a functionality perspective, Bulbasaur must be assigned 10 for its randomized seed, and Pikachu must be assigned a randomized seed of 11. Bulbasaur will have the first attack opportunity.

**Bulbasaur is attacking with Tackle for 1 damage to Pikachu
Pikachu has received 1 dmg, remaining hp is 24**

Once the battle has started, the Pokémon must take turns and communicate with each other as they attack and defend. Bulbasaur gets the first turn to attack or defend itself. Bulbasaur decides to attack and uses the Tackle attack – causing 1 hit point of damage – against Pikachu. Since this is the first action of the battle, Pikachu hasn't had an opportunity to defend itself, so the full damage value from the attack is inflicted on Pikachu. Both Pokémon begin with twenty-five (25) hit points, so Pikachu's current hit points are reduced to 24. Now, it's Pikachu's turn.

**Pikachu is attacking with Growl for 1 damage to Bulbasaur
Bulbasaur has received 1 dmg, remaining hp is 24**

Now Pikachu gets the opportunity to attack or defend, and decides to use the Growl attack, causing 1 hit point of damage against Bulbasaur. Since Bulbasaur chose to attack on its previous turn, and not defend itself, it will receive the full damage value from Pikachu's attack. Bulbasaur's hit points are reduced from 25 to 24. Now, it's Bulbasaur's turn.

**Bulbasaur is attempting to defend with Endure
Pikachu is attempting to defend with Endure
Bulbasaur is attempting to defend with Block
Pikachu is attempting to defend with Endure**

These actions represent four consecutive turns where Bulbasaur and Pikachu both decided to defend themselves. No damage occurs to either of them without an attack. Now, it's Bulbasaur's turn.

**Bulbasaur is attacking with Vine Whip for 2 damage to Pikachu
Pikachu successfully reduced Bulbasaur's damage by 1 with Endure
Pikachu has received 1 dmg, remaining hp is 23**

Bulbasaur has decided to use the Vine Whip attack, which normally causes 2 hit points of damage. Pikachu, however, used the Endure defense on its previous turn, so this reduces the damage by one point. Consequently, Pikachu's hit points are reduced by $(2 - 1) = 1$ point, from 24 down to 23. Now, it's Pikachu's turn.

**Pikachu is attempting to defend with Protect
Bulbasaur is attempting to defend with Block
Pikachu is attempting to defend with Protect
Bulbasaur is attempting to defend with Protect
Pikachu is attempting to defend with Endure
Bulbasaur is attempting to defend with Endure**

More defensive posturing from both Pokémon. Now, it's Pikachu's turn.

**Pikachu is attacking with Thunder Shock for 3 damage to Bulbasaur
Bulbasaur successfully reduced Pikachu's damage by 1 with Endure**

Bulbasaur has received 2 dmg, remaining hp is 22

Pikachu has decided to use the Thunder Shock attack, which normally causes 3 hit points of damage. Bulbasaur, however, used the Endure defense on its previous turn, so this reduces the damage by one point. Consequently, Pikachu's hit points are reduced by $(3 - 1) = 2$ points, from 24 down to 22.

...

Pikachu is attempting to defend with Protect

We resume the battle many turns later, with Bulbasaur at 2 hit points, and Pikachu at 10 hit points. It was Pikachu's turn, and Pikachu defended itself with Protect. Now, it's Bulbasaur's turn.

Bulbasaur is attacking with Razor Leaf for 3 damage to Pikachu

Pikachu successfully reduced Bulbasaur's damage by 3 with Protect

Pikachu has received 0 dmg, remaining hp is 10

Bulbasaur has decided to use the Razor Leaf attack, which normally causes 3 hit points of damage. Pikachu, however, used the Protect defense on its previous turn, so this reduces the damage by three points. Consequently, Pikachu's hit points aren't reduced at all. Now, it's Pikachu's turn.

Pikachu is attacking with Tail Whip for 2 damage to Bulbasaur

Bulbasaur has received 2 dmg, remaining hp is 0

Bulbasaur has lost

Pikachu has won the battle

Pikachu attacks Bulbasaur with a Tail Whip causing 2 hit points of damage. Bulbasaur didn't defend itself last turn, so the full 2 hit points from the attack impacts Bulbasaur, and its hit points are reduced to zero. Therefore, Pikachu wins the battle.

```
> // stop
```

```
> stop
```

```
stop acknowledged
```

```
simulation terminated
```

The stop command ends the simulation loop gracefully.

Closing Comments & Suggestions

This is the information that has been provided by the customer so far. We (the OMSCS 6310 Team) will likely conduct an Office Hours where you will be permitted to ask us questions to clarify the client's intent, etc. We will answer some of the questions, but we will not necessarily answer all of them. Also, though this current version of the system is "the core" of the system going forward, our clients might add, update, and possibly remove some requirements over the span of the course. One of your tasks will be to ensure that your architectural documents and related artifacts remain consistent with the problem requirements – and your system implementations – over time.

Quick Reminder on Collaborating with Others

Please use Ed Discussion for your questions and/or comments, and post publicly whenever it is appropriate. If your questions or comments contain information that specifically provides an answer for some part of the assignment, then please make your post private, and we will review it and decide if it is suitable to be shared with the rest of the class. Best of luck to you with this assignment.